StudentPostTests

Javadoc-style documentation draft for TP2 Task 6

Primary scope	Semi-automated suites	Manual suite
TP2 Tasks 5 and 6	PostTest, ReplyTest, DiscussionBoardDatabaseTest	Discussion-board UI workflow checks

Overview

This document records the complete set of semi-automated tests and manual tests used to show that the student discussion-board implementation satisfies the intent of the Students User Stories. The testing set is organized in the same spirit as Task 4 documentation: each test suite is treated as an important software component, documented by purpose, dependencies, rationale, supported operations, and the specific user-story behavior it verifies.

The semi-automated portion focuses on entity behavior and persistence behavior, where repeatable JUnit checks provide the strongest evidence. The manual portion focuses on JavaFX interaction paths that are driven by dialogs, selection state, search/filter widgets, and other view/controller behavior that is easier to verify by direct use of the application.

Recorded automated results at the time of preparation

Test class	Package	Primary purpose	Observed result
PostTest	entityClasses	Verifies Post construction, edit behavior, solved state, and formatted metadata	8/8 passing
ReplyTest	entityClasses	Verifies Reply construction, edit behavior, and formatted metadata	5/5 passing
DiscussionBoardDatabaseTest	database	Verifies post/reply persistence CRUD against the discussion-board database methods	8/8 passing
Manual discussion-board suite	guiDiscussionBoard	Verifies end-to-end UI/controller behavior that depends on dialogs, selection state, and table refresh behavior	To be completed with screenshots / recorded observations

Coverage summary towards Students User Stories

The matrix below shows how the documented test set collectively covers the required student-post behavior. Several requirements are intentionally covered by more than one test type: entity/database tests prove stable data behavior, while manual GUI tests prove that the same behavior is reachable and understandable in the running application.

Requirement / story intent	Semi-automated evidence	Manual evidence	Pass interpretation
Create post	DiscussionBoardDatabaseTest: createPostAndGetDiscussionPosts_persistsPost	Create post through UI	A new post is accepted, stored, and visible in the list/detail view.
Read/view stored posts	DiscussionBoardDatabaseTest retrieval checks after create/update/delete	Create post through UI; search/filter behavior	The user can view persisted posts and the displayed values match stored values.
Update post	PostTest editPost_* methods; DiscussionBoardDatabaseTest updatePost_persistsEditedHeaderBodyAndEditState	Edit post	Changed header/body values persist and edited metadata is updated.
Delete post	DiscussionBoardDatabaseTest deletePost_removesPostFromDiscussionPosts	Delete post with confirmation	Deleted posts are removed from list results only after confirmation.
Create reply	DiscussionBoardDatabaseTest createReplyAndGetPostReplies_persistsReplyUnderCorrectPost	Create reply	A reply is accepted, linked to the correct parent post, and visible beneath that post.
Update reply	ReplyTest editReply_updatesBodyAndMarksEdited; DiscussionBoardDatabaseTest updateReply_persistsEditedBodyAndEditState	Edit reply	Changed reply text persists and edited metadata is visible.
Delete reply	DiscussionBoardDatabaseTest deleteReply_removesReplyFromReplyMap	Delete reply	The reply is removed from the reply list for that parent post.
Solved / reopened state	PostTest markAsSolved_trueThenFalse_togglesSolvedState; DiscussionBoardDatabaseTest updatePostSolvedStatus_persistsSolvedFlag	Mark solved / reopen	Solved flag and UI label change correctly, then revert when reopened.
Input validation for blank content	PostTest and ReplyTest reject literal empty edits; DB tests provide persistence safety after valid input only	Invalid empty post; invalid empty reply	The application rejects invalid blank content and does not corrupt stored data.
Search / subset behavior	Not best suited to unit tests because filtering is view-driven	Search/filter behavior	Typing a keyword or changing solved status updates the visible subset

			correctly.
Selection-state warnings / safe UI behavior	Not best suited to unit tests because warnings depend on interactive selection state	Warning when no post is selected	The application shows a clear warning instead of failing when no table item is selected.

Package Summary

entityClasses tests

This package contains repeatable JUnit tests for the entity classes that model post and reply data. These tests are valuable because they isolate behavior that should remain correct regardless of the graphical interface: object construction, editing rules, solved-state toggling, and formatted metadata values.

database tests

This package contains repeatable JUnit tests for persistence behavior implemented by the Database class. These tests verify that the discussion-board data layer can create, read, update, and delete posts and replies, and that solved-state updates and reply-group deletion behave as intended.

guiDiscussionBoard manual tests

This package-level test suite is documented manually because the most important behaviors in this area depend on JavaFX dialogs, selection state, warnings, and view refresh behavior. The suite demonstrates that the same operations proven by the unit tests are actually reachable through the user interface and behave in a way a student can understand.

PostTest

Package: entityClasses

Purpose

PostTest documents and verifies the behavior of the Post entity class. Its purpose is to show that the Post model can safely represent a student discussion post, track edits, preserve solved status, and expose formatted values used by the view/controller layer.

Dependencies under test

- Post
- CreateInfo

Fixture / attribute rationale

Fixture element	Why it exists	Requirement impact
Fresh Post instances	Used to prove default construction and default state assumptions such as unsolved posts and unedited metadata.	Supports create/read expectations for student-authored posts.
Mapped / reconstructed Post instances	Used to prove that posts loaded from persistence still expose the correct values.	Supports compatibility between Task 3 entity design and Task 4 MVC/database usage.
Literal empty-string edits	Used because the current implementation performs direct empty-string checks.	Shows that invalid empty edit input is rejected without altering stored content.

Method summary

Test method	Intent	User-story / requirement coverage
constructorAndGetters_workForNewPost	Verifies a newly created post stores user, header, body, ID, and default flags correctly.	Create/read post behavior.
fullConstructor_preservesMappedValues	Verifies reconstruction from CreateInfo and mapped values.	Persistence/read compatibility.
editPost_updatesBothHeaderAndBody	Verifies valid edits update the main text and edit metadata.	Update post behavior.
editPost_updatesOnlyHeaderWhenBodyIsEmptyLiteral	Verifies partial edit behavior for header-only changes.	Update post behavior / validation.
editPost_updatesOnlyBodyWhenHeaderIsEmptyLiteral	Verifies partial edit behavior for body-only changes.	Update post behavior / validation.
editPost_returnsFalseWhenBothInputsAreEmptyLiterals	Verifies blank edit rejection without unintended mutation.	Input validation.
markAsSolved_trueThenFalse_togglesSolvedState	Verifies solved/reopened state transitions.	Solved / close / reopen behavior.
getCreateInfoFormatted_returnsCreateDateBeforeEdit_andExpandedAfterEdit	Verifies user-visible formatted metadata before and after edit.	Readability of displayed post history.

ReplyTest

Package: entityClasses

Purpose

ReplyTest documents and verifies the behavior of the Reply entity class. Its purpose is to show that replies carry the correct parent-post linkage, enforce safe edit rules, and provide the formatted data needed for list/detail displays.

Dependencies under test

- Reply
- CreateInfo

Fixture / attribute rationale

Fixture element	Why it exists	Requirement impact
Fresh Reply instances	Used to prove default construction and direct parent-post linkage.	Supports create/read reply behavior.
Mapped / reconstructed Reply instances	Used to prove that reply rows loaded from persistence still retain the expected values.	Supports compatibility between database retrieval and MVC display.
Literal empty-string edits	Used because the current implementation performs direct empty-string checks.	Shows invalid blank reply edits are rejected.

Method summary

Test method	Intent	User-story / requirement coverage
constructorAndGetters_workForNewReply	Verifies a new reply stores user, body, parent post ID, ID, and default edit state.	Create/read reply behavior.
fullConstructor_preservesMappedValues	Verifies reconstruction from CreateInfo and mapped values.	Persistence/read compatibility.
editReply_updatesBodyAndMarksEdited	Verifies valid reply edits update text and edit metadata.	Update reply behavior.
editReply_returnsFalseWhenBodyIsEmptyLiteral	Verifies blank edit rejection without unintended mutation.	Input validation.
getCreateInfoFormatted_returnsCreateDateBeforeEdit_andExpandedAfterEdit	Verifies user-visible formatted metadata before and after edit.	Readability of displayed reply history.

DiscussionBoardDatabaseTest

Package: database

Purpose

DiscussionBoardDatabaseTest documents and verifies persistence behavior for discussion-board posts and replies. It proves that the Database methods used by the MVC discussion-board code can actually store, retrieve, update, and delete the underlying entities required by the Students User Stories.

Dependencies under test

- Database
- Post
- Reply

Fixture / attribute rationale

Fixture element	Why it exists	Requirement impact
Dedicated Database instance per test	Provides a fresh connection and isolates each JUnit scenario.	Supports repeatable semi-automated evidence.
createdPosts / createdReplies tracking lists	Allow explicit cleanup after each run because the H2 configuration is persistent rather than throwaway.	Prevents test residue from weakening later results.
Unique suffix helper values	Ensure newly created posts and replies are distinguishable from pre-existing data.	Avoids false positives when reading back stored values.

Method summary

Test method	Intent	User-story / requirement coverage
createPostAndGetDiscussionPosts_persistsPost	Verifies create + retrieval for posts.	Create/read post behavior.
updatePost_persistsEditedHeaderBodyAndEditState	Verifies that edited post values and edit state persist.	Update post behavior.
updatePostSolvedStatus_persistsSolvedFlag	Verifies solved/reopened persistence.	Solved / reopen behavior.
deletePost_removesPostFromDiscussionPosts	Verifies post deletion from persisted list results.	Delete post behavior.
createReplyAndGetPostReplies_persistsReplyUnderCorrectPost	Verifies create + retrieval for replies under the correct parent post.	Create/read reply behavior.
updateReply_persistsEditedBodyAndEditState	Verifies persisted reply edits.	Update reply behavior.
deleteReply_removesReplyFromReplyMap	Verifies reply deletion from grouped retrieval results.	Delete reply behavior.
deletePostReplies_removesAllRepliesForOnePostOnly	Verifies targeted cleanup of replies for a selected post only.	Reply grouping / referential correctness.

DiscussionBoard – Manual Tests

Package: guiDiscussionBoard

Purpose

This manual suite verifies the interactive student workflow exposed by ViewDiscussionBoard and ControllerDiscussionBoard. These checks are necessary because several important behaviors depend on JavaFX dialogs, table selection state, pop-up warnings, and refresh behavior that are not naturally exercised by entity- or database-level unit tests.

Method summary / manual procedure set

Manual test ID	Scenario	Primary expected result	Why this test matters
MT-01	Create post through UI	A valid post is accepted and appears in the post list/detail view.	Proves the student can reach post creation through the GUI, not just through persistence methods.
MT-02	Invalid empty post	The application rejects blank content and preserves prior state.	Demonstrates visible input validation.
MT-03	Edit post	Updated post values appear after save and display edited metadata.	Shows the update workflow is reachable and understandable.
MT-04	Delete post with confirmation	Deletion occurs only after confirmation; canceled deletion leaves the post intact.	Shows safe destructive-action behavior.
MT-05	Create reply	A reply is accepted and shown under the correct post.	Proves parent-child discussion flow in the GUI.
MT-06	Invalid empty reply	Blank reply input is rejected with a clear message.	Demonstrates visible validation for reply content.
MT-07	Edit reply	Reply text updates and remains linked to the same parent post.	Shows update workflow for replies.
MT-08	Delete reply	The selected reply is removed from the reply list for that post.	Shows safe deletion and view refresh for replies.
MT-09	Mark solved / reopen	Solved label/status changes correctly, then reverts when reopened.	Demonstrates the close/reopen style workflow required by the discussion board.
MT-10	Search/filter behavior	The visible post subset changes correctly for keyword and solved-status filters.	Shows list/subset behavior required by the student stories.
MT-11	Warning when no post is selected	The application warns the user instead of failing when an action is attempted without a selection.	Demonstrates safe handling of selection-state edge cases.

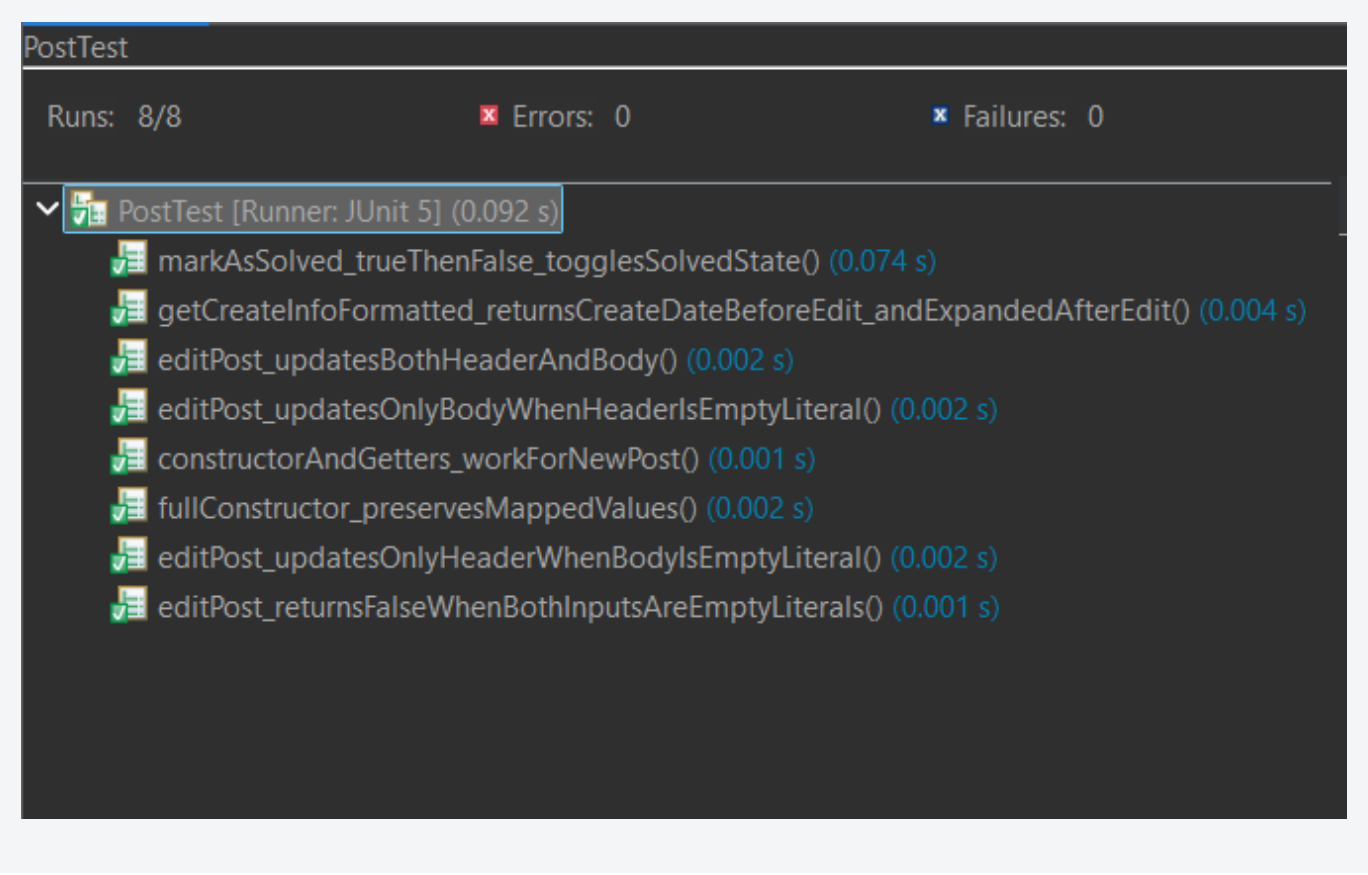
How to interpret output and decide pass/fail

Semi-automated tests are interpreted in the standard JUnit manner: each test class should compile, execute, and finish with a green result bar and zero failures. A passing test class demonstrates that the tested behavior remained correct for the exact conditions listed in the corresponding method summary. When a JUnit test fails, the documented behavior should be treated as not yet proven.

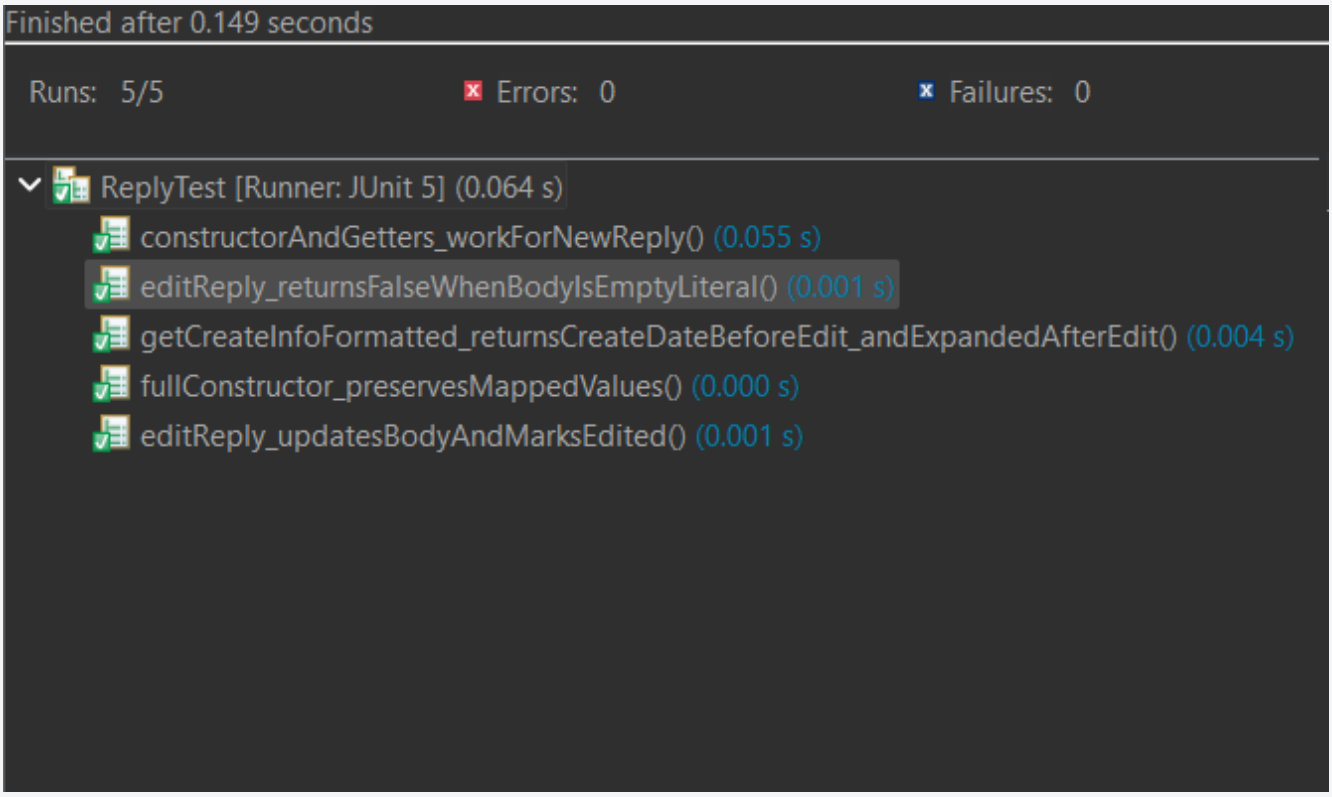
Manual tests are interpreted with the same discipline, but through observed application behavior rather than an assertion framework. Each manual test should record the starting state, the exact interaction performed, the visible result, and a pass/fail note. A manual test only counts as passing when the observed interface behavior matches the documented expected result and does not produce unintended errors, stale data, or confusing state changes.

Appendix A

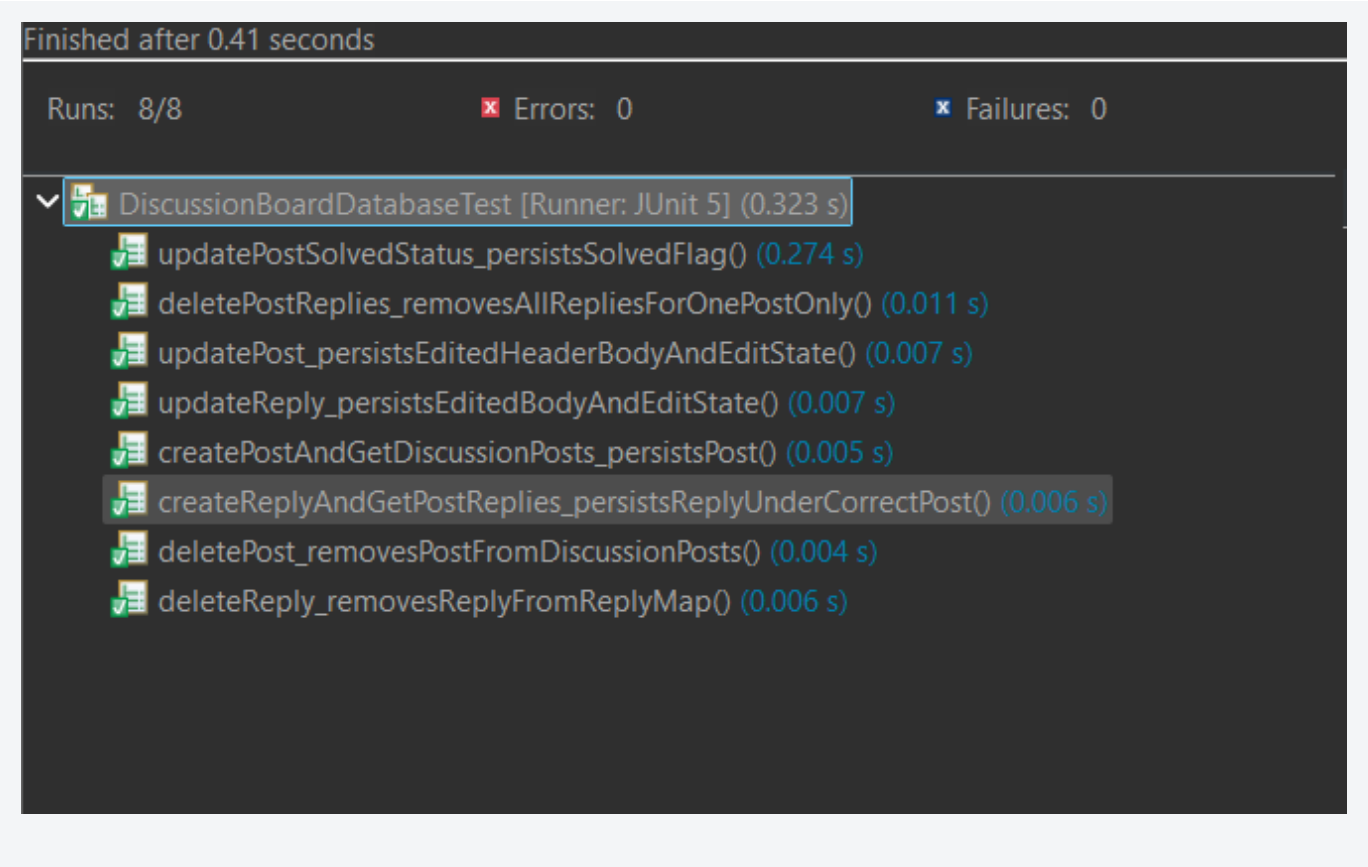
A-1 Automated evidence: PostTest result window



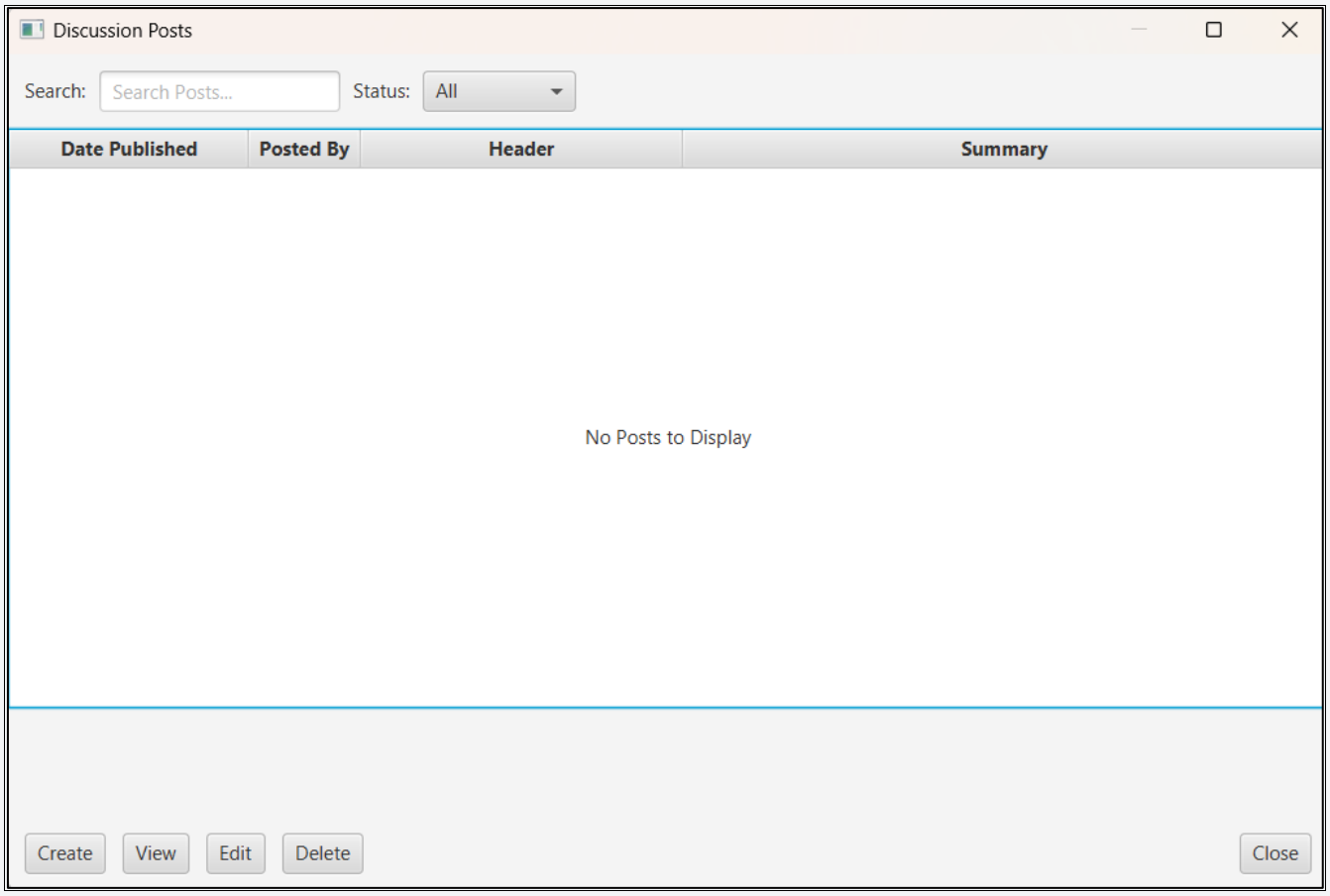
A-2 Automated evidence: ReplyTest result window

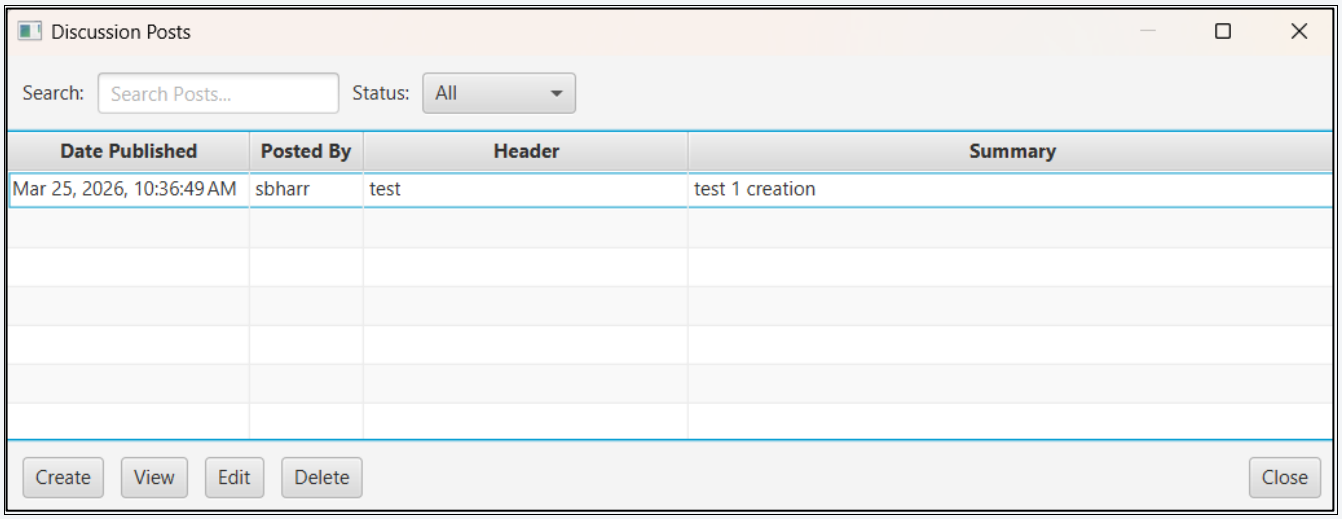


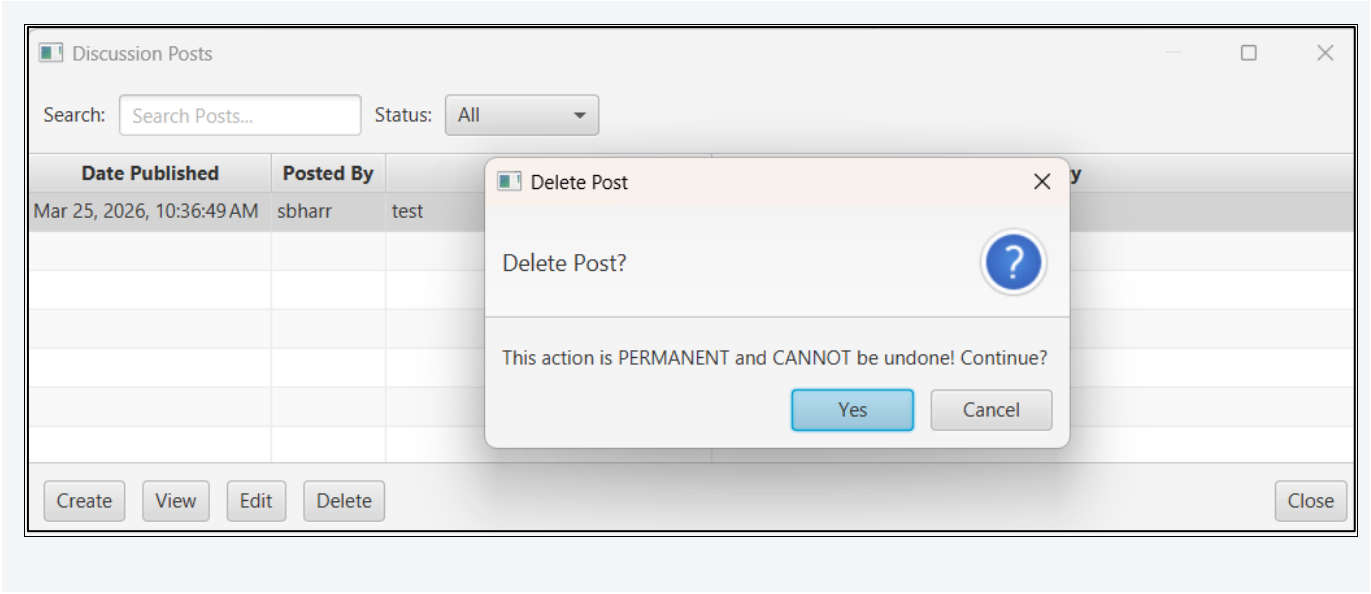
A-3 Automated evidence: DiscussionBoardDatabaseTest result window



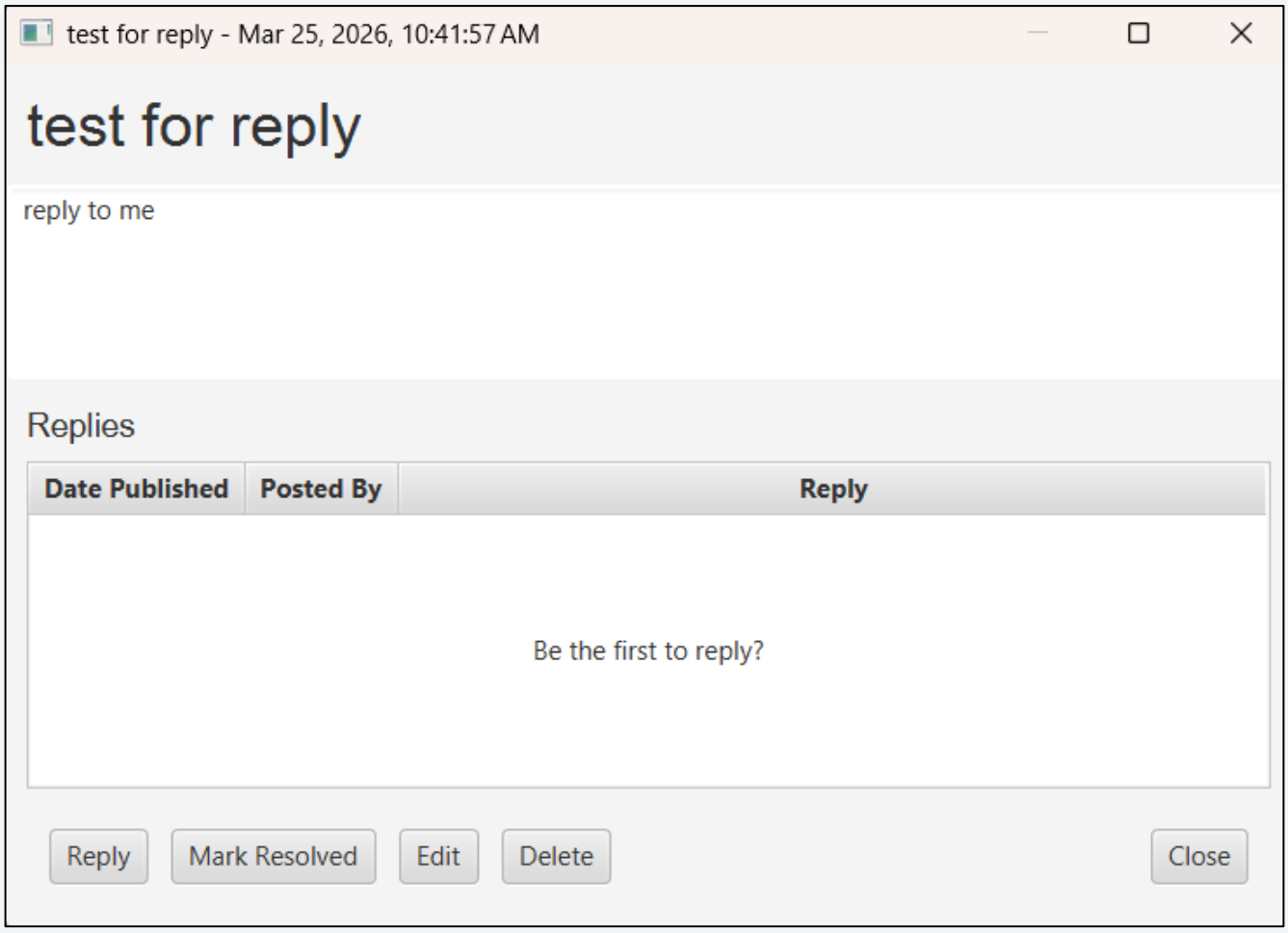
A-4 Manual evidence: create/edit/delete post flow

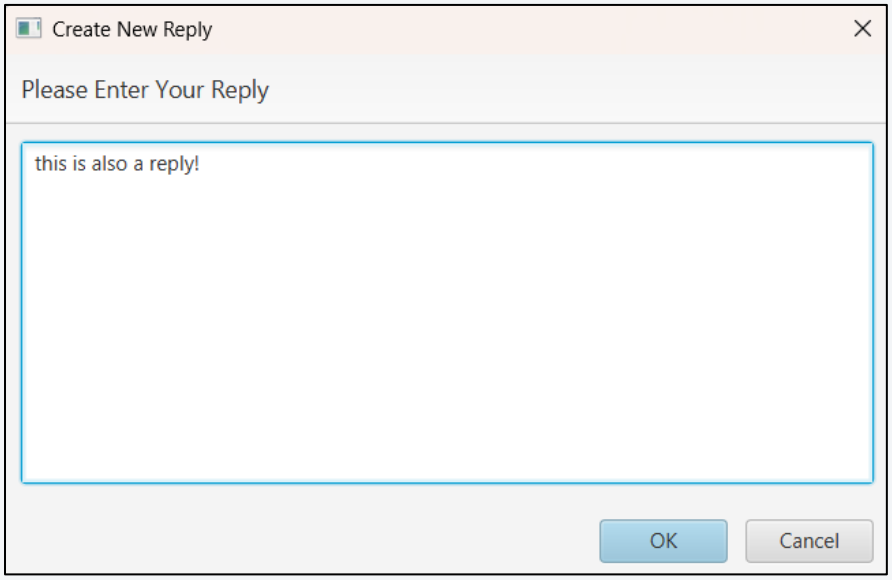


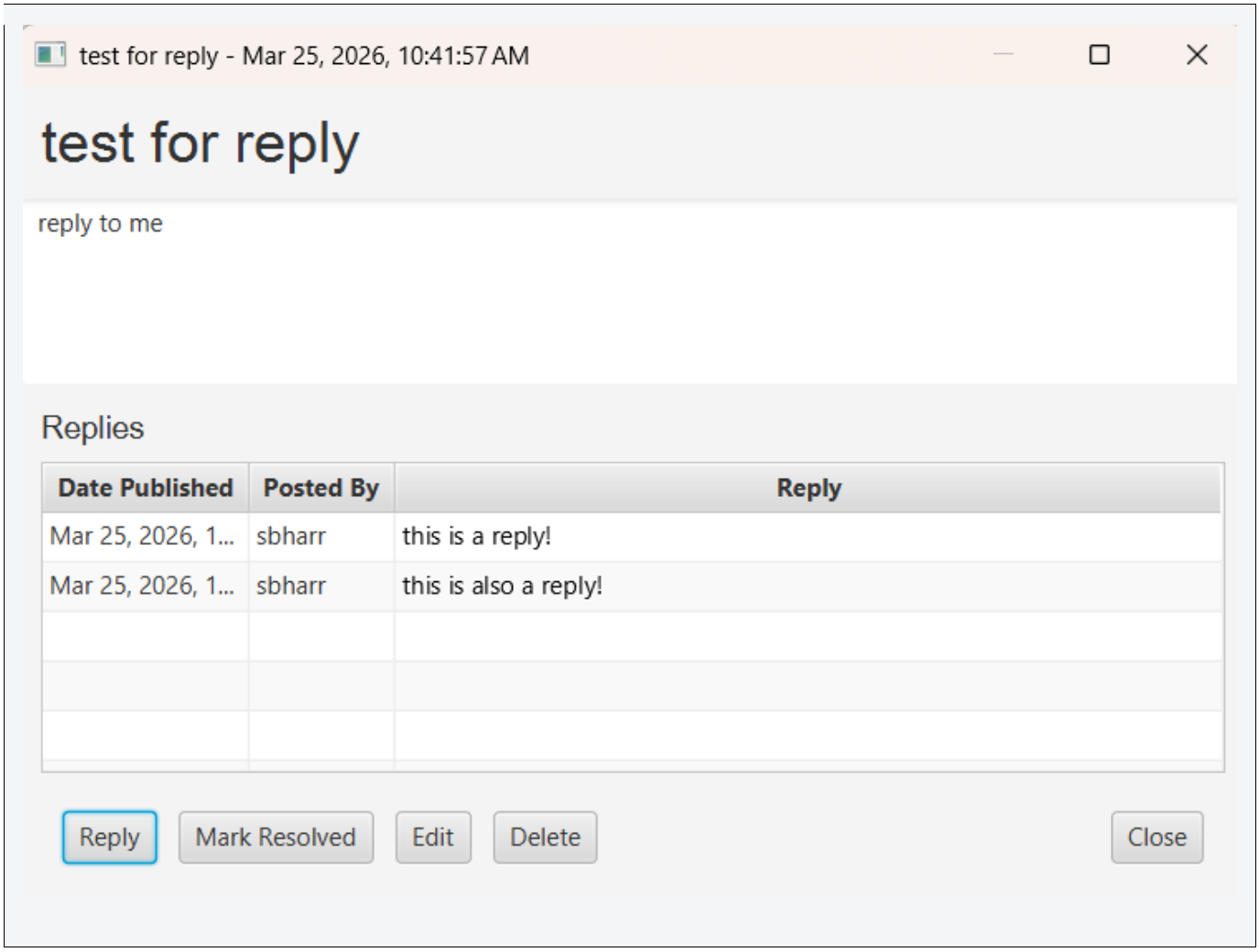



Discussion Posts
Search:
Search Posts...
Status:
All
Date Published
Posted By
Mar 25, 2026, 10:36:49 AM
sbharr
test
Delete Post
Delete Post?
This action is PERMANENT and CANNOT be undone! Continue?
Yes
Cancel
Create
View
Edit
Delete
Close

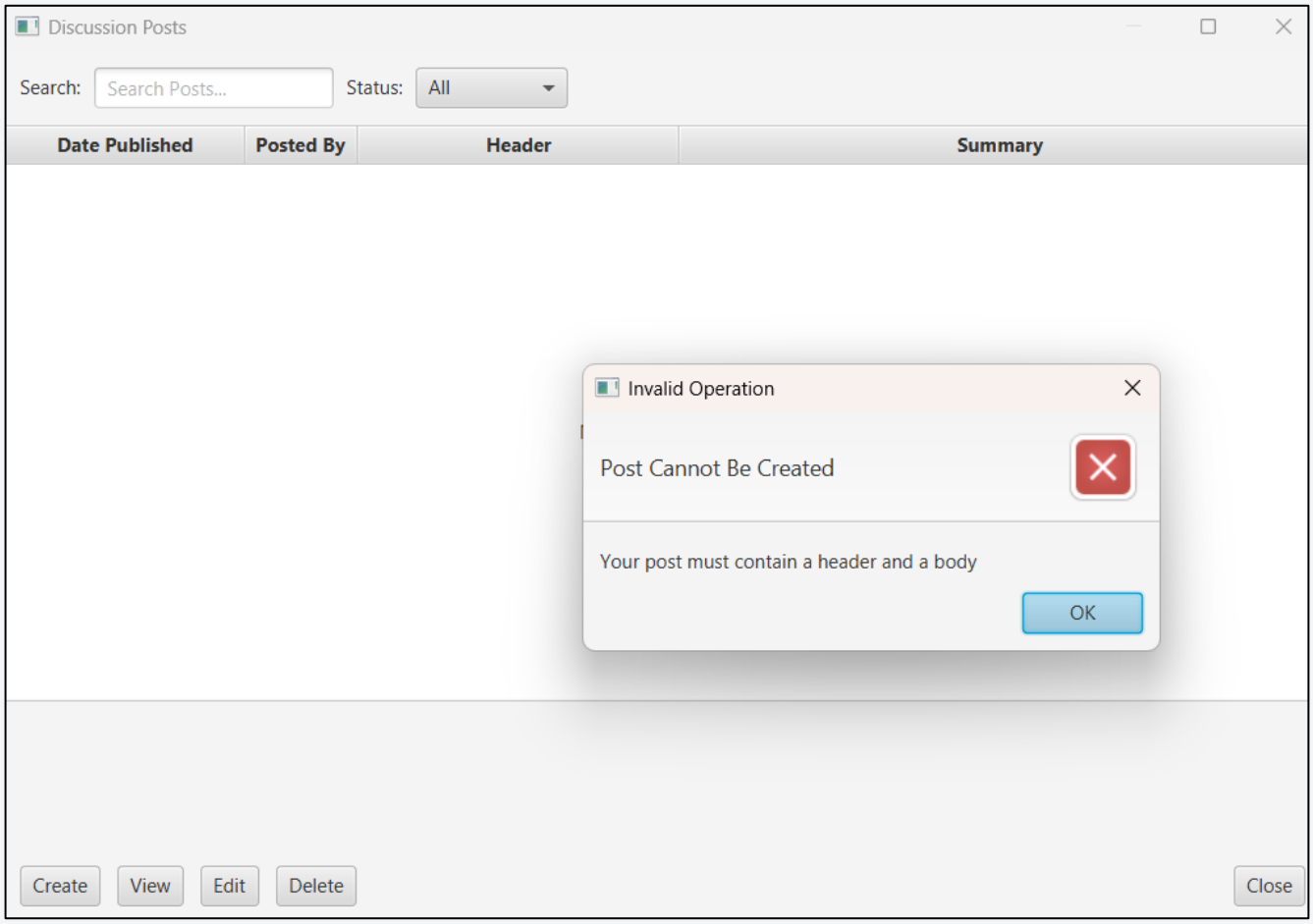
A-5 Manual evidence: create/edit/delete reply flow

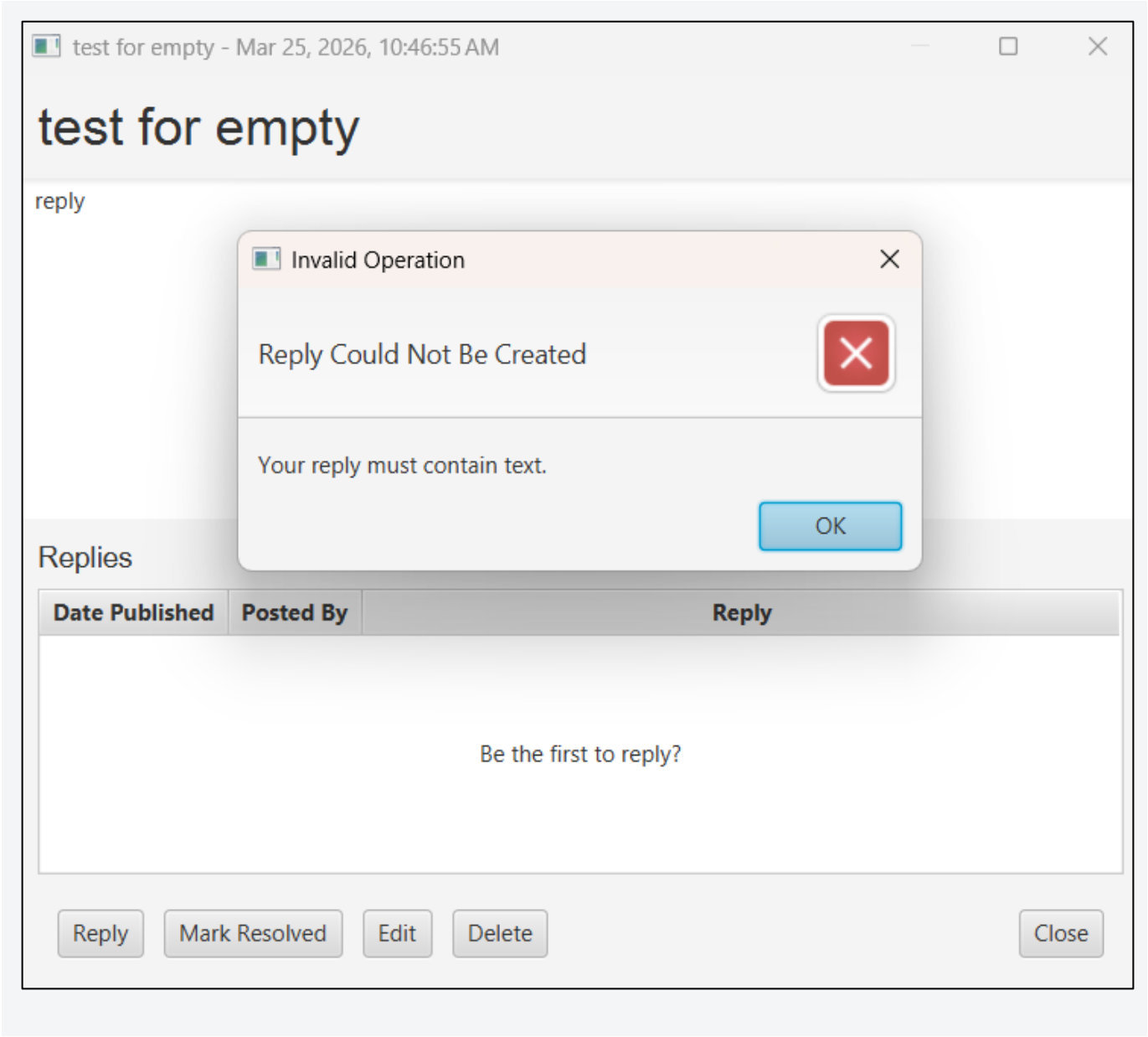




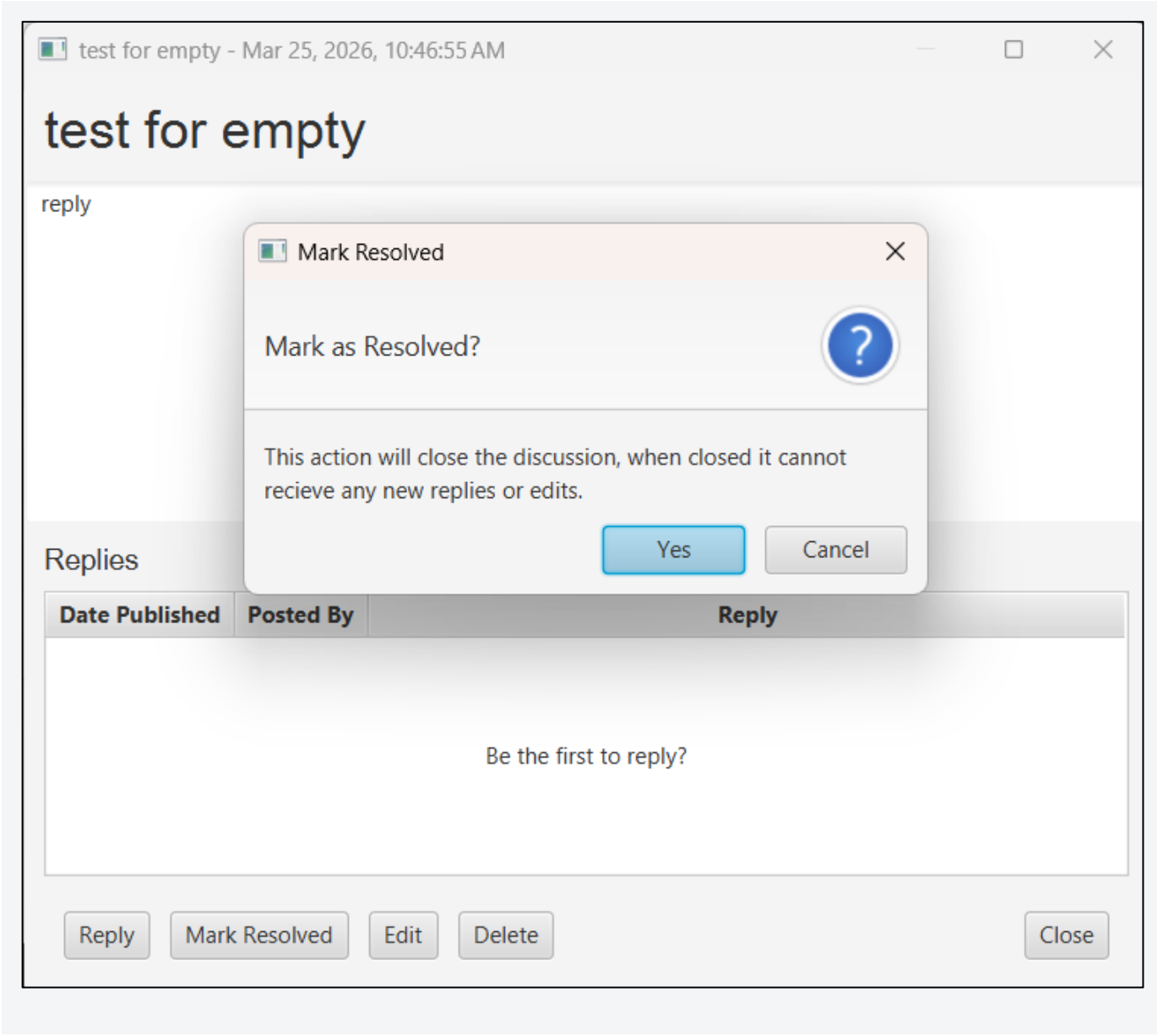

test for reply - Mar 25, 2026, 10:41:57 AM
test for reply
reply to me
Replies
Date Published
Posted By
Reply
Mar 25, 2026, 1...
sbharr
this is a reply!
Mar 25, 2026, 1...
sbharr
this is also a reply!
Reply
Mark Resolved
Edit
Delete
Close

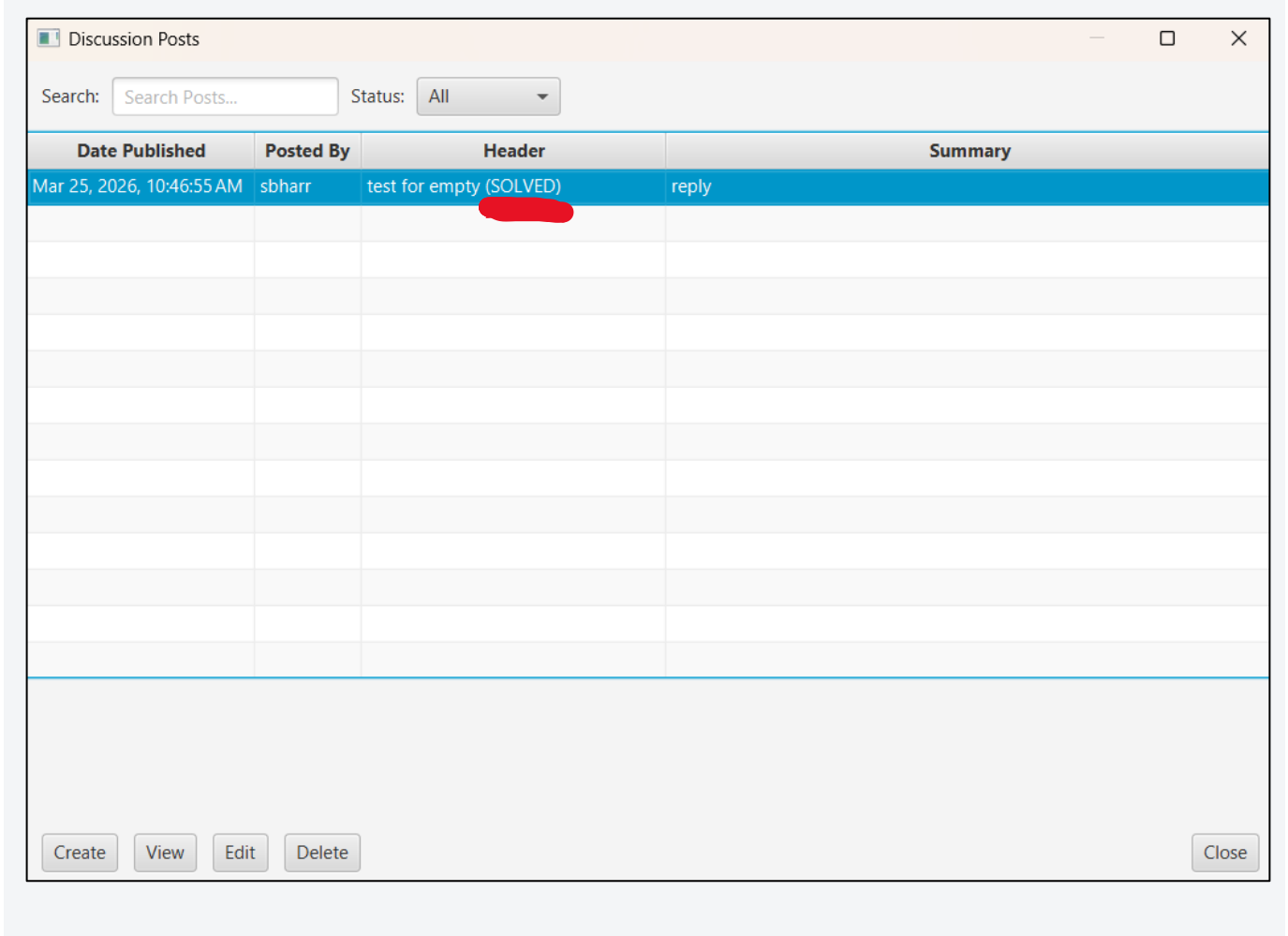
A-6 Manual evidence: validation and warnings

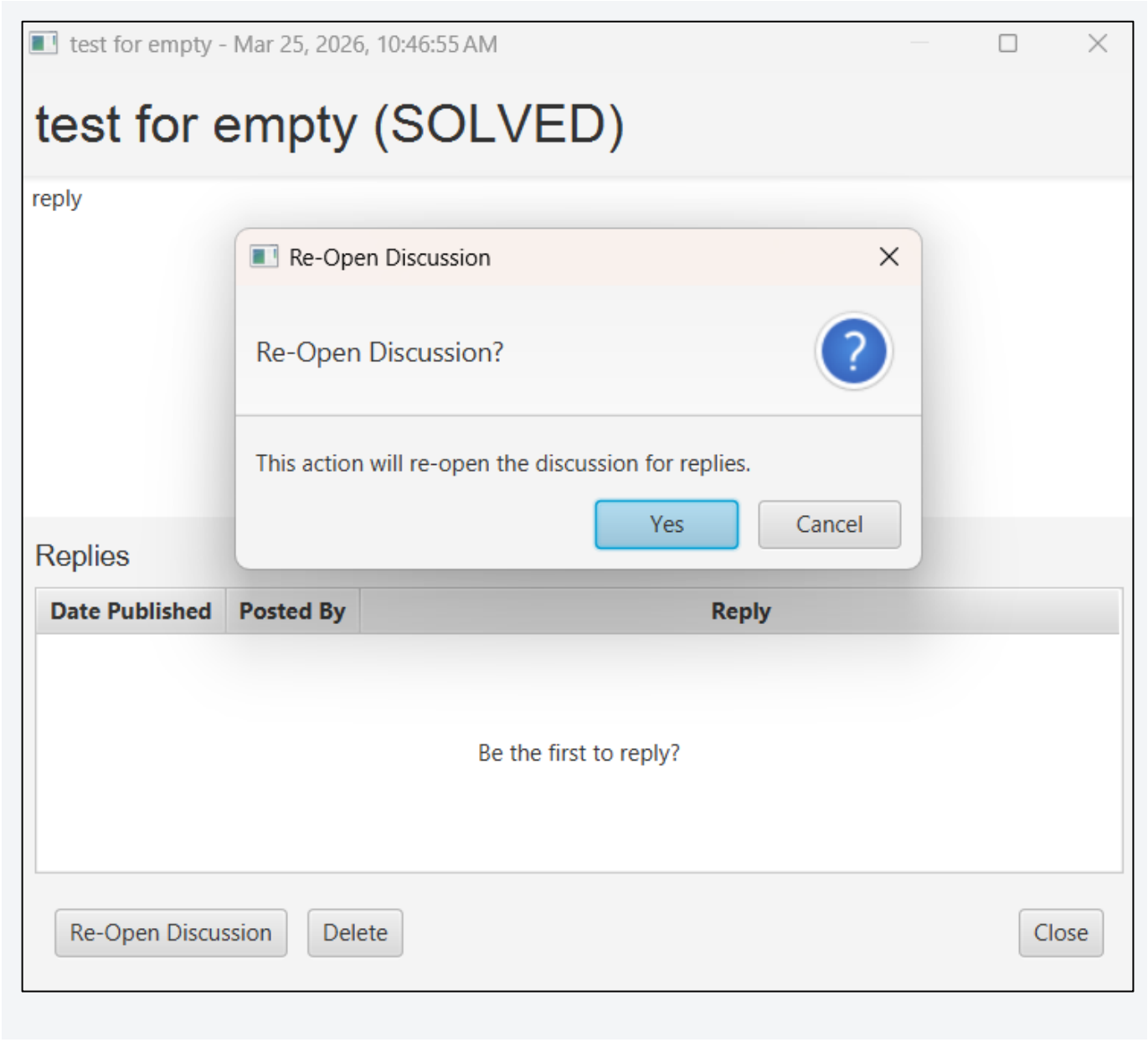



test for empty - Mar 25, 2026, 10:46:55 AM
test for empty
reply
Invalid Operation
Reply Could Not Be Created
Your reply must contain text.
OK
Replies
Date Published
Posted By
Reply
Be the first to reply?
Reply
Mark Resolved
Edit
Delete
Close

A-7 Manual evidence: solved + search/filter behavior




Discussion Posts
Search:
Search Posts...
Status:
All
Date Published
Posted By
Header
Summary
Mar 25, 2026, 10:46:55 AM
sbharr
test for empty (SOLVED)
reply
Create
View
Edit
Delete
Close


test for empty - Mar 25, 2026, 10:46:55 AM
test for empty (SOLVED)
reply
Re-Open Discussion
Re-Open Discussion?
This action will re-open the discussion for replies.
Yes
Cancel
Replies
Date Published
Posted By
Reply
Be the first to reply?
Re-Open Discussion
Delete
Close